

TECHNICAL REPORT: HOTEL RESERVATION SYSTEM

STUDENT INFORMATION

- **Name:** ONORE AKORTIA
- **Student ID:** MS/ITE/25/0035
- **Course:** INF811D - Object Oriented Programming
- **Program:** MSc Information Technology
- **Semester:** 1, 2025/2026 Academic Year
- **Topic:** Hotel Reservation System

1. INTRODUCTION

The Hotel Reservation Management System is a Java-based desktop application designed to automate and streamline the hotel booking process. The system provides an intuitive graphical user interface (GUI) built using JavaFX, enabling both guests and administrators to efficiently manage room reservations.

This project demonstrates the practical application of Object-Oriented Programming (OOP) principles in developing a real-world software solution. The system addresses common challenges in manual hotel management, such as double bookings, inefficient record-keeping, and lack of real-time availability information.

2. PROBLEM STATEMENT

Traditional hotel reservation methods face significant operational challenges:

1. **Manual Errors:** Paper-based systems are prone to double bookings, incorrect room assignments, and data entry mistakes.
2. **Inefficient Record Keeping:** Tracking reservations, guest histories, and room availability becomes increasingly difficult as operations grow.
3. **Real-time Availability:** Staff cannot instantly check room availability, leading to customer dissatisfaction.
4. **Administrative Burden:** Generating bills, managing room inventory, and maintaining guest records are time-consuming manual tasks.
5. **Data Redundancy:** Guest information is often duplicated across different records, leading to inconsistencies.

The Hotel Reservation Management System solves these problems by providing a centralized, automated platform for managing all hotel operations.

3. OBJECTIVES OF THE SYSTEM

Main Objective:

To develop a fully functional GUI-based Hotel Reservation Management System that demonstrates core Object-Oriented Programming principles.

Specific Objectives:

1. Implement a secure authentication system for different user roles (Guest and Administrator).
2. Provide an intuitive interface for guests to search available rooms, make bookings, and view reservation history.
3. Enable administrators to manage room inventory, view all reservations, and monitor system statistics.
4. Demonstrate the four pillars of OOP: Encapsulation, Inheritance, Polymorphism, and Abstraction.
5. Implement robust exception handling and input validation to ensure data integrity.
6. Create maintainable, well-documented code following software engineering best practices.

4. SCOPE OF THE SYSTEM**Features Included:**

Feature	Description
User Authentication	Login/Logout for Guests and Administrators
Room Management	View all rooms, check availability, add/remove rooms (Admin)
Room Booking	Select dates, choose room, calculate total price
Reservation Management	View and cancel reservations
Real-time Updates	Availability updates immediately after booking/cancellation
Input Validation	Date validation, room availability checks
Role-based Dashboards	Different interfaces for guests and admins

Features Excluded (Future Enhancements):

- Payment processing and online billing
- Email/SMS notification system
- Database integration (currently using in-memory storage)
- Mobile application version
- Advanced reporting and analytics
- Multi-language support

5. METHODOLOGY

The project was developed using a structured approach following software engineering best practices.

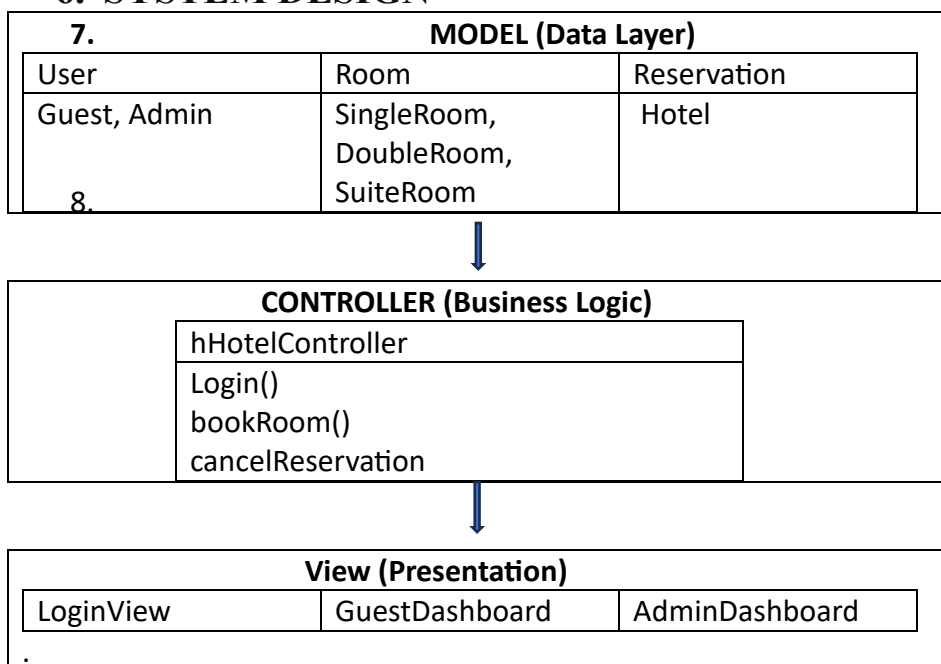
Development Phases:

- 1. Requirements Analysis**
 - Identified functional requirements for guests and administrators
 - Defined system features, constraints, and user stories
 - Established technical requirements (Java 17, JavaFX)
- 2. System Design**
 - Designed UML class diagrams showing relationships
 - Designed GUI prototypes for each screen
 - Defined the MVC (Model-View-Controller) architecture
- 3. Implementation**
 - Developed model classes with OOP principles
 - Implemented controller logic for business rules
 - Created JavaFX GUI components
 - Integrated event handling and validation
- 4. Testing**
 - Unit testing of individual classes and methods
 - Integration testing of complete workflows
 - User acceptance testing with sample scenarios
- 5. Deployment**
 - Source code hosted on GitHub
 - Comprehensive technical documentation
 - Application screenshots for demonstration

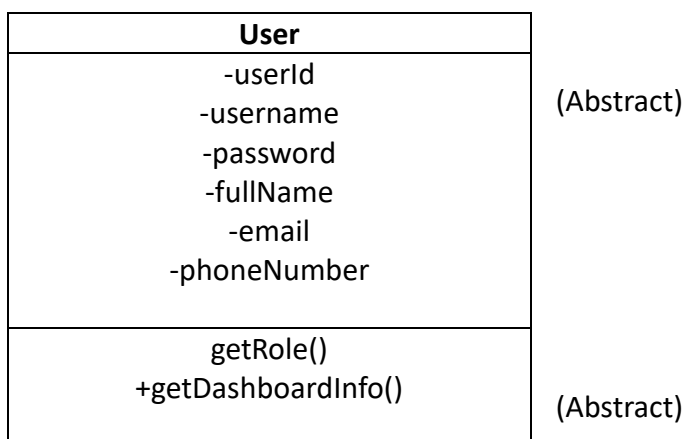
Tools and Technologies:

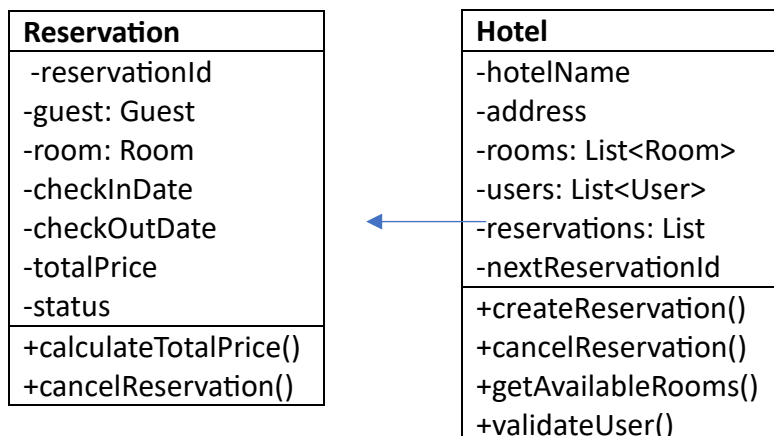
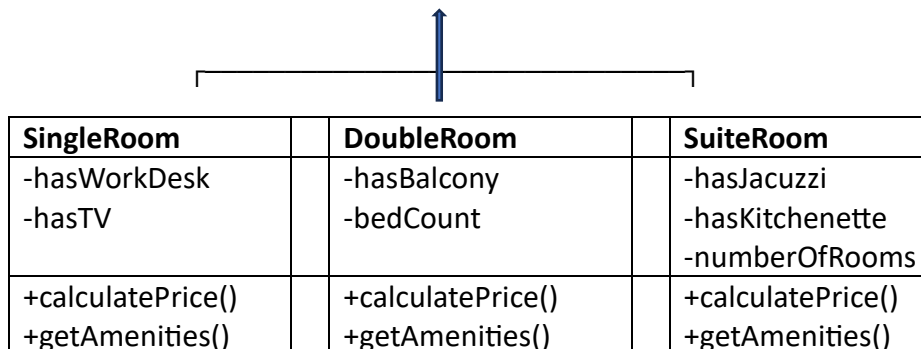
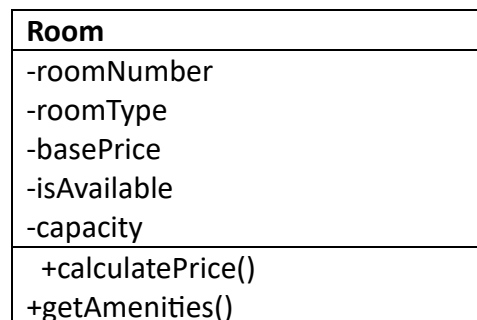
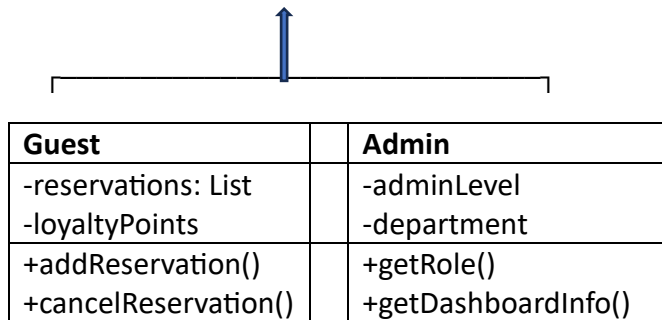
Tool	Purpose
Java 17	Programming Language
JavaFX	GUI Framework
IntelliJ IDEA	Development Environment
Git & GitHub	Version Control
Scene Builder	GUI Layout Design (optional)

6. SYSTEM DESIGN



6.2 Class Diagram





7. DESCRIPTION OF CLASSES AND METHODS

7.1 User (Abstract Class)

Purpose: Serves as the base class for all users in the system.

Fields:

```
java
private int userId;
private String username;
private String password;
private String fullName;
private String email;
private String phoneNumber;
```

Key Methods:

```
java
public abstract String getRole();    // Returns user role
public String getDashboardInfo() {   // Returns dashboard info
    return "Welcome " + fullName + " (" + getRole() + ")";
}
```

OOP Concept Demonstrated: Abstraction (abstract class with abstract method)

7.2 Guest (Subclass of User)

Purpose: Represents a hotel guest with reservation history and loyalty points.

Fields:

```
java
private List<Reservation> reservations;
private double loyaltyPoints;
```

Key Methods:

```
java
public void addReservation(Reservation reservation) {
    reservations.add(reservation);
    loyaltyPoints += reservation.getTotalPrice() * 0.05; // 5% loyalty
}

public void cancelReservation(Reservation reservation) {
    reservations.remove(reservation);
    loyaltyPoints -= reservation.getTotalPrice() * 0.05;
}
```

```
@Override
public String getRole() {
    return "Guest";
}
```

OOP Concept Demonstrated: Inheritance (extends User), Encapsulation (private fields)

7.3 Admin (Subclass of User)

Purpose: Represents a system administrator with elevated privileges.

Fields:

```
java
private String adminLevel;
private String department;
```

Key Methods:

```
java
@Override
public String getRole() {
    return "Administrator";
}
```

```
@Override
public String getDashboardInfo() {
    return "Admin: " + getFullName() + " | Department: " + department;
}
```

OOP Concept Demonstrated: Inheritance (extends User), Polymorphism (method overriding)

7.4 Room (Abstract Class)

Purpose: Base class for all room types in the hotel.

Fields:

```
java
private int roomNumber;
private String roomType;
private double basePrice;
private boolean isAvailable;
private int capacity;
```

Key Methods:

```
java
public abstract double calculatePrice(int numberOfNights);
public abstract String getAmenities();

public String toString() {
    return "Room " + roomNumber + " (" + roomType + ") - $" + basePrice +
        "/night" + (isAvailable ? "[Available]" : "[Booked]");
}
```

OOP Concept Demonstrated: Abstraction (abstract class with abstract methods)

7.5 SingleRoom (Subclass of Room)

Purpose: Represents a single room with optional work desk and TV.

Fields:

```
java
private boolean hasWorkDesk;
private boolean hasTV;
```

Key Methods:

```
java
@Override
public double calculatePrice(int numberOfNights) {
    double price = getBasePrice() * numberOfNights;
    if (hasWorkDesk) {
        price += 10 * numberOfNights; // Extra charge
    }
    return price;
}

@Override
public String getAmenities() {
    String amenities = "Single Bed, AC, Attached Bathroom";
    if (hasWorkDesk) amenities += ", Work Desk";
    if (hasTV) amenities += ", TV";
    return amenities;
}
```

OOP Concept Demonstrated: Inheritance (extends Room), Polymorphism (method overriding)

7.6 DoubleRoom (Subclass of Room)

Purpose: Represents a double room with optional balcony.

Fields:

```
java
private boolean hasBalcony;
private int bedCount;
```

Key Methods:

```
java
@Override
public double calculatePrice(int numberOfNights) {
    double price = getBasePrice() * numberOfNights;
    if (hasBalcony) {
        price += 20 * numberOfNights;
    }
    if (bedCount > 1) {
        price += 15 * numberOfNights;
    }
    return price;
}

@Override
public String getAmenities() {
    String amenities = "Double Bed(s) x" + bedCount + ", AC, Attached Bathroom";
    if (hasBalcony) amenities += ", Balcony";
    return amenities;
}
```

OOP Concept Demonstrated: Polymorphism (overridden calculatePrice method)

7.7 SuiteRoom (Subclass of Room)

Purpose: Represents a luxury suite with premium amenities.

Fields:

```
java
private boolean hasJacuzzi;
private boolean hasKitchenette;
private int numberOfRooms;
```

Key Methods:

java

@Override

```
public double calculatePrice(int numberOfNights) {  
    double price = getBasePrice() * numberOfNights;  
    if (hasJacuzzi) price += 50 * numberOfNights;  
    if (hasKitchenette) price += 30 * numberOfNights;  
    price += (numberOfRooms - 1) * 25 * numberOfNights;  
    return price;  
}
```

@Override

```
public String getAmenities() {  
    String amenities = "Suite with " + numberOfRooms + " rooms, Premium AC, Luxury Bathroom";  
    if (hasJacuzzi) amenities += ", Jacuzzi";  
    if (hasKitchenette) amenities += ", Kitchenette";  
    return amenities;  
}
```

OOP Concept Demonstrated: Polymorphism (overridden calculatePrice and getAmenities)

7.8 Reservation

Purpose: Represents a room booking made by a guest.

Fields:

java

```
private int reservationId;  
private Guest guest;  
private Room room;  
private LocalDate checkInDate;  
private LocalDate checkOutDate;  
private double totalPrice;  
private String status; // "Confirmed", "Checked-in", "Checked-out", "Cancelled"
```

Key Methods:

java

```
public int getNumberOfNights() {  
    return (int) ChronoUnit.DAYS.between(checkInDate, checkOutDate);  
}  
  
public void calculateTotalPrice() {  
    int nights = getNumberOfNights();
```

```

    if (nights <= 0) {
        throw new IllegalArgumentException("Invalid date range");
    }
    this.totalPrice = room.calculatePrice(nights);
}

public void cancelReservation() {
    if ("Confirmed".equals(status) || "Checked-in".equals(status)) {
        this.status = "Cancelled";
        room.setAvailable(true);
        guest.cancelReservation(this);
    }
}
}

```

OOP Concept Demonstrated: Encapsulation (private fields with validation), Exception Handling

7.9 Hotel

Purpose: Central data management class that manages rooms, users, and reservations.

Fields:

```

java
private String hotelName;
private String address;
private List<Room> rooms;
private List<User> users;
private List<Reservation> reservations;
private Map<Integer, Guest> guestMap;
private int nextReservationId;

```

Key Methods:

```

java
public Reservation createReservation(Guest guest, Room room,
                                     LocalDate checkIn, LocalDate checkOut) {
    if (!room.isAvailable()) {
        throw new IllegalStateException("Room is not available");
    }
    if (checkIn.isBefore(LocalDate.now())) {
        throw new IllegalArgumentException("Check-in cannot be in the past");
    }
    if (checkOut.isBefore(checkIn) || checkOut.equals(checkIn)) {

```

```

        throw new IllegalArgumentException("Invalid check-out date");
    }

    Reservation reservation = new Reservation(nextReservationId++, guest, room,
        checkIn, checkOut);
    room.setAvailable(false);
    guest.addReservation(reservation);
    reservations.add(reservation);
    return reservation;
}

public List<Room> getAvailableRooms() {
    return rooms.stream().filter(Room::isAvailable).collect(Collectors.toList());
}

public boolean validateUser(String username, String password) {
    User user = findUserByUsername(username);
    return user != null && user.getPassword().equals(password);
}

```

OOP Concept Demonstrated: Encapsulation (private fields), Exception Handling (validation)

7.10 HotelController

Purpose: Manages business logic and acts as the bridge between Model and View.

Fields:

```

java
private Hotel hotel;
private User currentUser;

```

Key Methods:

```

java
public boolean login(String username, String password) {
    if (hotel.validateUser(username, password)) {
        currentUser = hotel.findUserByUsername(username);
        return true;
    }
    return false;
}

```

```

public Reservation bookRoom(int roomNumber, LocalDate checkIn, LocalDate checkOut) {
    if (!isGuest()) {
        throw new IllegalStateException("Only guests can book rooms");
    }
    Room room = hotel.findRoomByNumber(roomNumber);
    if (room == null) {
        throw new IllegalArgumentException("Room not found");
    }
    Guest guest = (Guest) currentUser;
    return hotel.createReservation(guest, room, checkIn, checkOut);
}

public void cancelReservation(int reservationId) {
    hotel.cancelReservation(reservationId);
}

public ObservableList<Reservation> getCurrentUserReservations() {
    if (isGuest()) {
        Guest guest = (Guest) currentUser;
        return FXCollections.observableArrayList(
            hotel.getGuestReservations(guest)
        );
    }
    return FXCollections.observableArrayList(hotel.getAllReservations());
}

```

OOP Concept Demonstrated: Encapsulation, Exception Handling, Role-based logic

8. GUI DESIGN EXPLANATION

8.1 Design Principles

Principle	Implementation
User-Friendly	Clear labels, intuitive navigation, helpful tooltips
Consistent	Uniform color scheme, fonts, and layouts across screens

Responsive	Immediate feedback on user actions
Professional	Clean, modern appearance with proper spacing

8.2 Color Scheme

Color	Hex Code	Purpose
Primary Blue	#2E86C1	Headers, primary buttons
Success Green	#27AE60	Confirmation actions
Danger Red	#E74C3C	Warnings, deletions
Background	White/Light Gray	Main interface

8.3 Screen Layouts

Login Screen:

- Title: "Hotel Reservation System"
- Username and Password fields
- Login button
- Demo credentials link

Guest Dashboard:

- Header: User greeting + Logout button
- Two tabs: "Book Room" and "My Reservations"
- Book Room: Room dropdown, date pickers, book button
- Reservations: Table view with cancel option

Admin Dashboard:

- Header: User greeting + Logout button
- Statistics bar (Total rooms, available, booked, reservations)
- Two tabs: "Room Management" and "All Reservations"

9. OOP CONCEPTS IMPLEMENTED

9.1 Encapsulation

Definition: Bundling data with methods that operate on that data, restricting direct access.

Implementation:

All class fields are declared `private` with public getters/setters.

Code Example:

```
java
public class User {
    private String username; // Private field
    private String password; // Private field

    // Public getters and setters
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

Benefits:

- Data hiding and security
- Controlled access to sensitive information
- Maintainability (changes only affect getter/setter methods)

9.2 Inheritance

Definition: Creating new classes based on existing classes to promote code reuse.

Implementation:

- `Guest` and `Admin` extend `User`
- `SingleRoom`, `DoubleRoom`, `SuiteRoom` extend `Room`

Code Example:

```
java
public class Guest extends User {
    // Inherits all fields and methods from User
}
```

```

private List<Reservation> reservations; // Guest-specific fields

public Guest(int userId, String username, String password,
             String fullName, String email, String phoneNumber) {
    super(userId, username, password, fullName, email, phoneNumber);
    this.reservations = new ArrayList<>();
}
}

```

Benefits:

- Code reusability
- Hierarchical relationships
- Extensible system design

9.3 Polymorphism

Definition: Objects of different classes respond to the same method call differently.

Implementation:

- calculatePrice() overridden in each room type
- getRole() overridden in Guest and Admin
- getDashboardInfo() overridden in subclasses

Code Example:

```

java
// In SingleRoom
@Override
public double calculatePrice(int nights) {
    return getBasePrice() * nights + 10; // Add work desk charge
}

// In DoubleRoom
@Override
public double calculatePrice(int nights) {
    return getBasePrice() * nights + 20; // Add balcony charge
}

// In SuiteRoom
@Override
public double calculatePrice(int nights) {
    return getBasePrice() * nights + 50; // Add jacuzzi charge
}

```


Benefits:

- Flexible and extensible code
- Single interface for multiple implementations
- Runtime method binding

9.4 Abstraction

Definition: Hiding complex implementation details and showing only essential features.

Implementation:

- `User` is an abstract class with abstract `getRole()` method
- `Room` is an abstract class with abstract `calculatePrice()` and `getAmenities()` methods

Code Example:

```
java
public abstract class User {
    // Common fields and concrete methods
    private String fullName;

    public String getFullName() { return fullName; }

    // Abstract method - must be implemented by subclasses
    public abstract String getRole();
}

public abstract class Room {
    // Abstract methods - must be implemented by subclasses
    public abstract double calculatePrice(int nights);
    public abstract String getAmenities();
}
```

Benefits:

- Clear contracts for subclasses
- Reduced complexity
- Focus on essential features

10. EXCEPTION HANDLING

The system implements comprehensive exception handling at all user interaction points.

Validation Rules:

Validation	Implementation
Login	Empty username/password check
Date Validation	Check-in not in past, Check-out after check-in
Room Availability	Cannot book unavailable room
Booking	Guest must be logged in

Code Examples:

```

java
// Login validation
if (username.isEmpty() || password.isEmpty()) {
    statusLabel.setText("Please enter username and password");
    return;
}

try {
    if (controller.login(username, password)) {
        // Login successful
    }
} catch (Exception ex) {
    statusLabel.setText("Error: " + ex.getMessage());
}

// Booking validation
if (checkIn.isBefore(LocalDate.now())) {
    showAlert("Error", "Check-in date cannot be in the past");
    return;
}

if (checkOut.isBefore(checkIn) || checkOut.equals(checkIn)) {
    showAlert("Error", "Check-out must be after check-in");
    return;
}

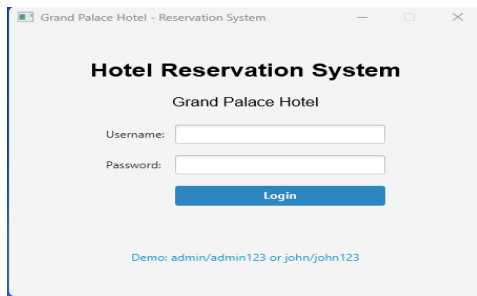
try {

```

```
Reservation reservation = controller.bookRoom(roomNumber, checkIn, checkOut);  
// Booking successful  
} catch (IllegalStateException | IllegalArgumentException ex) {  
    showAlert("Error", ex.getMessage());  
}
```

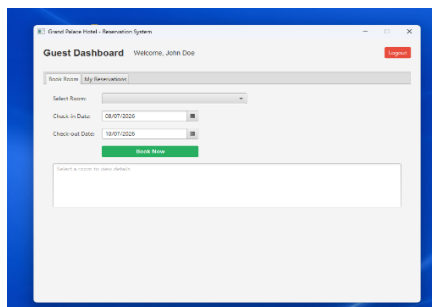
11. SCREENSHOTS AND OUTPUTS

11.1 Login Screen



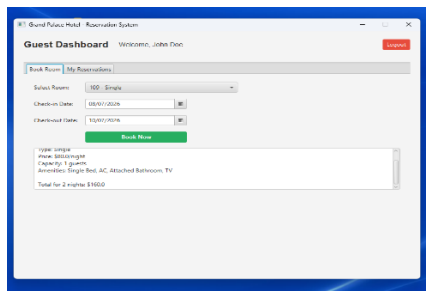
The login screen displays the application title, username and password fields, login button, and demo credentials information.

11.2 Guest Dashboard



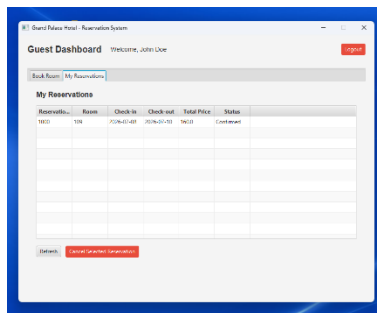
The guest dashboard shows a greeting message, logout button, and two tabs for booking rooms and viewing reservations.

11.3 Room Booking



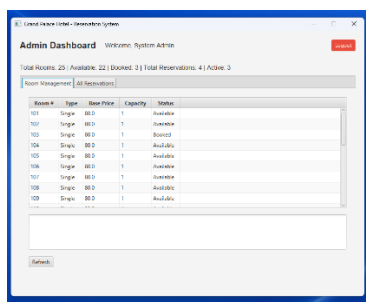
The booking tab displays available rooms in a dropdown, date pickers for check-in/check-out, and room details with price calculation.

11.4 Guest Reservations



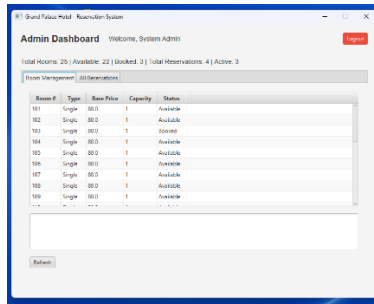
The reservations tab shows a table with reservation ID, room number, dates, total price, and status, with an option to cancel selected reservations.

11.5 Admin Dashboard



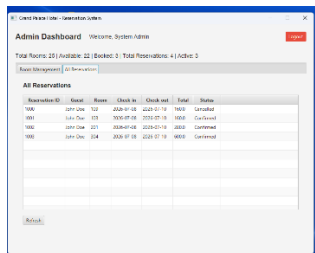
The admin dashboard displays system statistics and tabs for room management and all reservations.

11.6 Room Management (Admin)



The room management tab shows all rooms with their details and availability status.

11.7 All Reservations (Admin)



The all reservations tab displays all reservations in the system with guest names, room numbers, dates, and status.

11.8 Sample Outputs

Successful Booking:

text

Room booked successfully!

Reservation ID: 1001

Total: \$240.0

Cancellation Confirmation:

text

Are you sure you want to cancel this reservation?

[OK] [Cancel]

Validation Error:

text

Error: Check-in date cannot be in the past

12. GITHUB REPOSITORY LINK

Repository URL:

<https://github.com/ONORE-AKORTIA/hotel-reservation-system>

Repository Contents:

hotel-reservation-system/

```
|— src/
|   |— com/
|   |   |— hotel/
|   |       |— Main.java
|   |       |— model/
|   |           |— User.java
|   |           |— Guest.java
|   |           |— Admin.java
|   |           |— Room.java
|   |           |— SingleRoom.java
|   |           |— DoubleRoom.java
|   |           |— SuiteRoom.java
|   |           |— Reservation.java
|   |           |— Hotel.java
|   |       |— controller/
|   |           |— HotelController.java
|   |       |— view/
|   |           |— LoginView.java
|   |           |— GuestDashboardView.java
|   |           |— AdminDashboardView.java
|— screenshots/
|   |— login.png
|   |— guest-dashboard.png
|   |— guest-booking.png
|   |— guest-reservations.png
|   |— admin-dashboard.png
|   |— admin-rooms.png
|   |— admin-reservations.png
|— README.md
|— .gitignore
```

Commit History:

The repository shows incremental commits demonstrating the development process:

1. Initial project setup with folder structure
2. Implementation of model classes (User, Guest, Admin)
3. Implementation of room classes (Room, SingleRoom, DoubleRoom, SuiteRoom)

4. Implementation of Hotel and Reservation classes
5. Implementation of HotelController
6. Implementation of GUI views (Login, Guest Dashboard, Admin Dashboard)
7. Adding exception handling and validation
8. Adding screenshots and documentation

13. CHALLENGES ENCOUNTERED

13.1 Technical Challenges

Challenge	Solution
JavaFX Setup	Added JavaFX libraries to classpath and module path
Date Validation	Used LocalDate API with comprehensive validation logic
Polymorphic Methods	Carefully overrode methods in each subclass
Real-time UI Updates	Used ObservableList and property bindings
Exception Handling	Implemented try-catch blocks with user-friendly messages

13.2 Development Challenges

Challenge	Solution
Time Management	Created development schedule with phase milestones
Understanding OOP Concepts	Studied examples and ensured clear demonstration of each principle

Testing All Scenarios	Created comprehensive test plan
GUI Consistency	Used CSS for consistent styling across all screens

14. CONCLUSION

The Hotel Reservation Management System has been successfully developed as a fully functional GUI-based Java application. The system meets all specified requirements and demonstrates a strong understanding of Object-Oriented Programming principles.

Key Achievements:

1. **OOP Principles:** Successfully implemented Encapsulation (private fields with getters/setters), Inheritance (Guest/Admin extend User, room types extend Room), Polymorphism (overridden calculatePrice methods), and Abstraction (abstract User and Room classes).
2. **GUI Design:** Created a professional, user-friendly interface with clear navigation and consistent styling.
3. **Functionality:** Implemented complete reservation management with authentication, room booking, cancellation, and real-time availability updates.
4. **Validation:** Comprehensive input validation and exception handling at all interaction points.
5. **Documentation:** Well-structured, commented code with complete technical documentation.

The project has provided valuable hands-on experience in Java programming, GUI development, and software engineering practices applicable to future software development projects.

15. RECOMMENDATIONS

For Future Enhancement:

Recommendation	Description
Database Integration	Replace in-memory storage with MySQL/PostgreSQL
Payment Processing	Integrate Stripe/PayPal for online payments

Email Notifications	Send confirmation emails and reminders
Mobile Application	Develop mobile version for on-the-go booking
Advanced Reports	Generate occupancy and revenue analytics
Security Enhancement	Implement password hashing and session management
Multi-language	Add support for multiple languages
Online Check-in	Allow guests to check-in online

16. REFERENCES

1. Horstmann, C. S. (2019). *Core Java Volume I - Fundamentals* (11th ed.). Pearson Education.
2. Schildt, H. (2018). *Java: The Complete Reference* (11th ed.). McGraw-Hill Education.
3. Oracle Corporation. (2023). *JavaFX Documentation*. <https://openjfx.io/>
4. Freeman, E., & Freeman, E. (2020). *Head First Design Patterns* (2nd ed.). O'Reilly Media.
5. GitHub Documentation. (2024). *About READMEs*. <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>
6. University of Cape Coast. (2025). *INF811D: Object Oriented Programming Course Materials*.
7. TutorialsPoint. (2024). *Java Tutorial*. <https://www.tutorialspoint.com/java/index.htm>
8. GeeksforGeeks. (2024). *JavaFX Tutorial*. <https://www.geeksforgeeks.org/javafx-tutorial/>
9. Martin, R. C. (2018). *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson Education.
10. Java Community. (2024). *Java Documentation*. <https://docs.oracle.com/en/java/>

STUDENT DECLARATION

I, **ONORE AKORTIA** (Student ID: **MS/ITE/25/0035**), hereby declare that this technical report and the accompanying project work is my original work and has not been submitted previously for any academic award.

Signature: _____

Date: _____

INSTRUCTOR COMMENTS

Marks:

Section	Marks Obtained
Functionality of System (20)	
GUI Design and User Experience (15)	
Application of OOP Concepts (20)	
Operators and Control Structures (10)	
Methods and Modularization (10)	
Event Handling and GUI Interaction (10)	
Exception Handling and Validation (5)	
GitHub Deployment and Documentation (5)	
Technical Report (5)	
Total (100)	

Comments:

Instructor's Signature: _____

Date: _____